ApplyMate: A Unified AI-Driven Platform for Automated Scholarship and Job Applications

FYP Code: 23F-13

Software Requirement Specification

BY

HAMADULLAH SAMO, CMS. ID: 023-23-0041

MARYAM KHATOON, CMS. ID: 053-23-0021

HASSAN ALI, CMS. ID: 023-23-0046

Supervised by: Dr Saif Hassan _____

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

NOVEMBER 2026

Document Revision History

Version	Date	Description of change	Author(s)
1.0	06 Nov 2026	First version of the SRS, submitted to the supervisor for review. Later versions will follow the supervisor's suggestions.	All members

1. INTRODUCTION....1

1.1. Purpose of Document....1
1.2. Intended Audience....1
1.3. Document Convention, abbreviations, and acronyms....1
2. OVERALL SYSTEM DESCRIPTION....3

2.1. Project Background....3
2.2. Project Scope....3
2.3. Not in Scope....4
2.4. Project Objectives....4
2.5. Stakeholders....5
2.6. Operating Environment....6
2.7. System Constraints....6
2.8. Assumptions & Dependencies....7
3. EXTERNAL INTERFACE REQUIREMENTS....8

3.1. Hardware Interfaces....8
3.2. Software Interfaces....8
3.3. Communications Interfaces....9
4. FUNCTIONAL REQUIREMENTS....11

4.1. Functional Hierarchy....11
4.2. Use Cases....13
4.2.1. UC-01: Sign Up....15
4.2.2. UC-02: Log In....15
4.2.3. UC-03: Manage Profile....16
4.2.4. UC-04: Import Profile from CV....17
4.2.5. UC-05: Upload Documents....17
4.2.6. UC-06: Browse Matched Opportunities....18
4.2.7. UC-07: View Opportunity Details....19
4.2.8. UC-08: Add Job by Link....20
4.2.9. UC-09: Apply with Agent....21
4.2.10. UC-10: Respond to Agent Pause....22
4.2.11. UC-11: Review and Submit Application....23
4.2.12. UC-12: Track Applications....24
4.2.13. UC-13: View Notifications and Reminders....24
4.2.14. UC-14: Ask ApplyMate Chatbot....25
4.2.15. UC-15: Teach Route to Application Form....26
4.2.16. UC-16: Reset Forgotten Password....27
4.2.17. UC-17: Answer Form Questions....27
4.2.18. UC-18: Prepare Before Applying....28
5. NON-FUNCTIONAL REQUIREMENTS....30

5.1. Performance Requirements....30
5.2. Safety Requirements....30
5.3. Security Requirements....31
5.4. User Documentation....32
6. REFERENCES....33

7. APPENDICES....34

1. Introduction

1.1. Purpose of Document

This Software Requirements Specification (SRS) describes what ApplyMate must do and the conditions it must work under. ApplyMate is our Final Year Project (FYP 23F-13): a web platform where a student builds one profile, sees the scholarships and jobs that actually suit that profile, and lets an AI agent fill the real application form on the provider's own website.

We wrote this document so that everyone involved agrees on the same picture of the system before and during development. It records the scope, the users, the external systems ApplyMate talks to, the functional requirements as use cases, and the non-functional requirements such as privacy, safety and performance. The design decisions that satisfy these requirements are covered separately in our Software Design Specification (SDS).

The requirements here reflect the system as it stands after Mid-Defense (November 2026). They include what we learned by running the agent on real Erasmus Mundus application portals: the questions a form asks are collected once, AI answers are checked before they are typed, and the student sees open, upcoming and closed scholarships at a glance. A small number of requirements are marked as planned; we will complete them before the final defense.

1.2. Intended Audience

We expect the following people to read this document:

- Our supervisor, who reviews the requirements and checks that the project has enough depth for a final year project.
- The FYP evaluation committee, who use the SRS at Mid-Defense and Final Defense to judge whether the delivered system meets what we promised.
- Our team (Hamadullah Samo, Maryam Khatoon and Hassan Ali), who use it as the reference while building and testing each module.
- Future students or developers who may extend ApplyMate, for example by adding new scholarship portals or job sites.

Readers who only need an overview can read Sections 1 and 2. Section 4 is the most useful part for anyone checking behaviour in detail, and Section 5 lists the quality rules the system must respect.

1.3. Document Convention, abbreviations, and acronyms

The body text of this document uses Times New Roman, 12 pt, justified. Main headings are 14 pt bold and sub-headings 12 pt bold, numbered automatically. Tables use 11 pt text with a shaded header row. Requirement identifiers follow a fixed pattern: FR-x.y for functional requirements (module x, requirement y), UC-xx for use cases, and NFR-P, NFR-S and NFR-SEC for performance, safety and security requirements. The words "must" and "shall" mark a mandatory requirement, while "should" marks a desirable one.

The abbreviations and acronyms used in this document are listed below.

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CAPTCHA	Test used by websites to tell humans and automated programs apart
CNIC	Computerized National Identity Card (Pakistan)
CV	Curriculum Vitae
DAAD	German Academic Exchange Service (Deutscher Akademischer Austauschdienst)
DOM	Document Object Model, the structure of a web page that a browser builds
FR / NFR	Functional Requirement / Non-functional Requirement
FYP	Final Year Project
HEC	Higher Education Commission of Pakistan
JWT	JSON Web Token, used to keep a user signed in
LLM	Large Language Model
MERN	MongoDB, Express, React and Node.js
OCR	Optical Character Recognition (reading text from scanned images)
OTP	One-Time Password or verification code
RAG	Retrieval-Augmented Generation: the model answers only from records retrieved for the question
REST	Representational State Transfer, the style of our web API
SDS / SRS	Software Design Specification / Software Requirements Specification
SH	Stipendium Hungaricum, the Hungarian Government scholarship
SMTP	Simple Mail Transfer Protocol, used to send email
UC	Use Case
UI	User Interface

A few project terms also appear often. A "portal" is the external website where an application is filled, such as the HEC Scholarship Portal. A "route" is the sequence of clicks that leads from a portal's start page to its application form. A "pause" is a moment when the agent stops and hands control to the student, for example to type a password or write an essay.

2. Overall System Description

2.1. Project Background

Every year thousands of Pakistani students apply for international scholarships such as Fulbright, Chevening, DAAD, Erasmus Mundus and Stipendium Hungaricum, and for jobs on sites like Rozee.pk. Each provider runs its own website with its own form, field names, date formats, dropdowns and document rules. A student who applies to five programmes types the same name, CNIC, address, education history and marks five times, on five different layouts.

We saw three problems while planning this project. First, information is scattered: students hear about scholarships through social media posts and often find out about a programme after its deadline, or apply to one they are not eligible for. Second, the forms are long and inconsistent, so students make small mistakes, such as a wrong date format or a missing degree record, that can get an application rejected. Third, many students give up half way, especially on portals with multi-step forms, pop-ups and pages that are hard to navigate.

Existing tools only solve part of this. Scholarship listing websites help with discovery but leave the form filling to the student. Browser autofill handles a name and an email but cannot understand a dropdown called "Domicile" or a pop-up for adding a Matric record. General web agents from research, such as those evaluated in Mind2Web and WebArena, show that language models can operate websites, but they usually depend on paid cloud models and are not built around a student's private data or a "never submit for me" rule.

ApplyMate started as ScholarHub AI, a scholarship discovery platform. Our supervisor felt that discovery alone was too small for a final year project, and asked for an agent that takes the student's data from the platform and fills it into the actual scholarship websites. That request shaped the system described here.

2.2. Project Scope

ApplyMate is a web platform for students of any department, not only computer science. Its boundaries are set by four main functions.

One profile. The student keeps one complete profile: personal details, address, education history from Matric to Master's, languages, statements, work experience, preferences and documents. The profile can be started by importing a CV in PDF or Word format, including scanned CVs through OCR.

Suitable opportunities only. ApplyMate keeps a curated catalogue of official scholarship programmes (Fulbright, Chevening, Stipendium Hungaricum through HEC, several Erasmus Mundus joint masters and several DAAD programmes) and checks each programme's status. The scholarships page is a simple board with three columns: open now, opening soon, and deadline passed. Each listing is matched against the student's level, field of study and stated requirements such as work experience, so the student sees the opportunities that suit them, with a fit score and clear reasons. Jobs can be added by pasting a link from Rozee.pk or another job site.

A details page for every opportunity. The system reads the programme's own website and shows the requirements, documents to prepare, deadlines, funding and how to apply. A "Before you apply" checklist shows which documents and details the student still has to add, and the questions this programme's form asked in earlier runs, so they can be answered before the agent starts.

An AI agent that applies with the student. When the student clicks "Apply with agent" and gives consent, the agent opens the official website in a browser, finds the application form by itself, detects each field type, fills it from the profile and moves through the form step by step. It stops and asks the student only when that is truly necessary: to log in, enter a code, solve a CAPTCHA, write an essay, accept a declaration, confirm an upload on a real website, or press the final submit button. Questions the profile cannot answer are asked inside ApplyMate in plain words, and every answer is saved so that no form asks it twice.

Around these functions the system also provides an application tracker, deadline reminders, in-app and email notifications, account settings with password reset by email, and "Ask ApplyMate", a chatbot that answers questions about the listed opportunities. All AI processing runs on an offline model (qwen3:4b through Ollama) on the student's own machine or our server, so personal data does not go to a cloud AI service.

2.3. Not in Scope

To keep the project realistic and ethical, we have deliberately left the following out:

- Submitting applications. The agent never presses the final submit button. The student always reviews and submits personally.
- Solving CAPTCHAs or getting around bot protection such as Cloudflare. When a site blocks automated browsing, as Rozee.pk does, ApplyMate respects it and offers another way, such as adding the job by link.
- Storing or typing the student's passwords for external portals, or reading verification codes from the student's email or phone.
- Writing essays, motivation letters, abstracts or statements of purpose on the student's behalf. Several portals state that they do not accept AI-written motivation. The agent only pastes text the student has saved, and asks for anything new.
- Making choices that belong to the student, such as ranking universities, choosing a track or specialisation, or saying how they heard about a programme. The agent always asks.
- Paying application fees or entering card or bank details.
- Guaranteeing admission or a scholarship. ApplyMate prepares the application. The provider makes the decision.
- Covering every scholarship in the world. The catalogue is a fixed, curated set of international programmes relevant to Pakistani students, which the team extends over time.
- Native mobile applications. ApplyMate is a responsive web application. The agent runs on a laptop or desktop because it needs a full browser.

2.4. Project Objectives

Our objectives describe the result we want to reach by the final defense:

- Give every student one complete, reusable profile, so the same information never has to be typed twice.
- Show each student only the opportunities that suit their own profile, from any department, with an honest fit score and the reason when something does not suit them.

- Present each opportunity's requirements, documents, deadlines and application steps on one clear details page, read from the official source.
- Build our own agent that can reach the real application form on a real portal and fill it correctly, including difficult widgets such as searchable dropdowns, date pickers, icon radio buttons, pop-up dialogs and "Add Education" lists, without being hard-coded for one website.
- Keep the student in control: the agent pauses for anything personal or irreversible, and never submits.
- Ask each question only once: answers to questions a form asks are saved and reused on every later form, and the questions a programme asks are shown before the student applies.
- Never put wrong data into a real form: every answer the AI suggests is checked against the question before it is typed.
- Protect privacy by running the language model offline on the student's own computer, and by never giving it identifiers such as the CNIC, passport number, phone or email.
- Help students keep track of their applications and deadlines through a tracker, reminders and notifications.
- Demonstrate the full flow on at least two to three real scholarship platforms open to Pakistani students.

2.5. Stakeholders

The stakeholders of ApplyMate, and how each one is involved, are listed below.

Stakeholder	Role and interest
Students (primary users)	Undergraduate and graduate students of any department who want scholarships or jobs. They build the profile, browse opportunities, start the agent and answer its pauses.
ApplyMate team	Hamadullah Samo, Maryam Khatoon and Hassan Ali. We develop, test and maintain the system, keep the scholarship catalogue current, and record the route to a programme's application form once so the agent can follow it for every student.
Supervisor	Guides the project, reviews the SRS and SDS, and approves the scope and evaluation.
FYP evaluation committee	Assess the project at Mid-Defense and Final Defense against this document.
Scholarship and job providers	HEC, DAAD, the Tempus Public Foundation (Stipendium Hungaricum), Chevening, USEFP (Fulbright), Erasmus Mundus consortia, and job sites. They do not use ApplyMate directly, but their websites are the systems ApplyMate interacts with, and their terms of use limit what the agent may do.
Sukkur IBA University	The department that hosts the project and benefits from a tool its own students can use.

2.6. Operating Environment

ApplyMate runs as a web application with a separate agent process, in the following environment:

Component	Environment
Client	Any modern browser (Microsoft Edge, Google Chrome or Firefox) on a laptop, desktop or phone. Built with React 18 and Vite 5.
Server	Node.js (version 20 or later) running an Express 4 REST API, on Windows 10/11, Linux or macOS.
Database	MongoDB 6 or later, accessed through Mongoose 8.
AI agent	Node.js process using Playwright to control Microsoft Edge (or Chrome) with its own browser profile, so the student can log in to portals once and stay logged in.
Language model	Ollama running the qwen3:4b model locally (default address localhost:11434).
Other services	Tesseract.js for OCR of scanned CVs, and an SMTP account (for example Gmail with an app password) for email notifications.
Network	Internet access is needed to reach scholarship and job websites. The language model and database work without internet.

The recommended machine for running the full system, including the offline model and the agent's browser, is a laptop with at least 8 GB of RAM (16 GB preferred), a recent multi-core processor and about 10 GB of free disk space. A GPU speeds up the model but is not required.

2.7. System Constraints

The following constraints come from outside the system and shape how it must be built:

Software constraints. The system must use free or open-source components. The language model must run offline, which limits us to small models such as qwen3:4b that fit on a student laptop. Each portal is built with a different framework (Angular Material, select2, iframes, shadow DOM), so the agent has to work from what the browser shows rather than from a fixed list of field names.

Hardware constraints. The agent and the model need a laptop or desktop with enough memory; a phone can use the platform but cannot run the agent. A slower machine makes each language-model call take longer.

Cultural and language constraints. The interface and all portals we target are in English. Pakistani data formats must be supported, such as CNIC numbers, Matric and Intermediate boards, marks out of a total, CGPA out of 4.0, and domicile.

Legal and ethical constraints. The agent must respect each website's terms of use and anti-bot measures, must not solve CAPTCHAs, and must never submit an application or accept a legal declaration on the student's behalf. Personal data such as CNIC numbers and documents must be handled carefully and never shared with third parties or cloud AI services.

Environmental constraints. The agent depends on external websites that can be slow, change their layout, go down for maintenance, or close a scholarship call outside its cycle. The system must cope with these situations by pausing and explaining, rather than failing silently.

User constraints. Users are students of any department and are not expected to be technical. Messages must be in plain language, and the student must always know what the agent is doing and why it has stopped.

Off-the-shelf components. Playwright, Ollama, Tesseract.js and Nodemailer bring their own limits: for example, Playwright needs an installed Edge or Chrome, OCR accuracy depends on scan quality, and Gmail limits how many emails can be sent per day.

Academic constraints. The project must be completed by a team of three within two semesters, with a working demonstration on real portals at Mid-Defense on 5 November 2026.

2.8. Assumptions & Dependencies

We have made the following assumptions:

- The student enters truthful information and keeps the profile up to date. ApplyMate fills forms with what the profile says.
- The student already has, or can create, an account on each external portal, and logs in personally when the agent asks.
- Official programme websites remain publicly reachable and keep their general structure between cycles. When a route changes, the team can record it again.
- The student runs the agent on a machine that meets the operating environment above.

The system depends on these external factors:

- The availability and opening dates of scholarship calls. For example, a portal may let a student fill the profile but block the next step until the call is active.
- Third-party software: Ollama and the qwen3:4b model, Playwright with Microsoft Edge or Chrome, MongoDB, Tesseract.js and an SMTP provider.
- Internet connectivity for reaching portals and programme websites.
- The terms of use of each provider, which may restrict automated access at any time.

3. External Interface Requirements

ApplyMate connects to the people and systems shown in the context diagram below. The student and the ApplyMate team interact with the platform directly. On the other side, the platform reads programme websites, drives a browser on application portals, asks the offline language model for help with field meanings, stores data in MongoDB and sends email through an SMTP service.

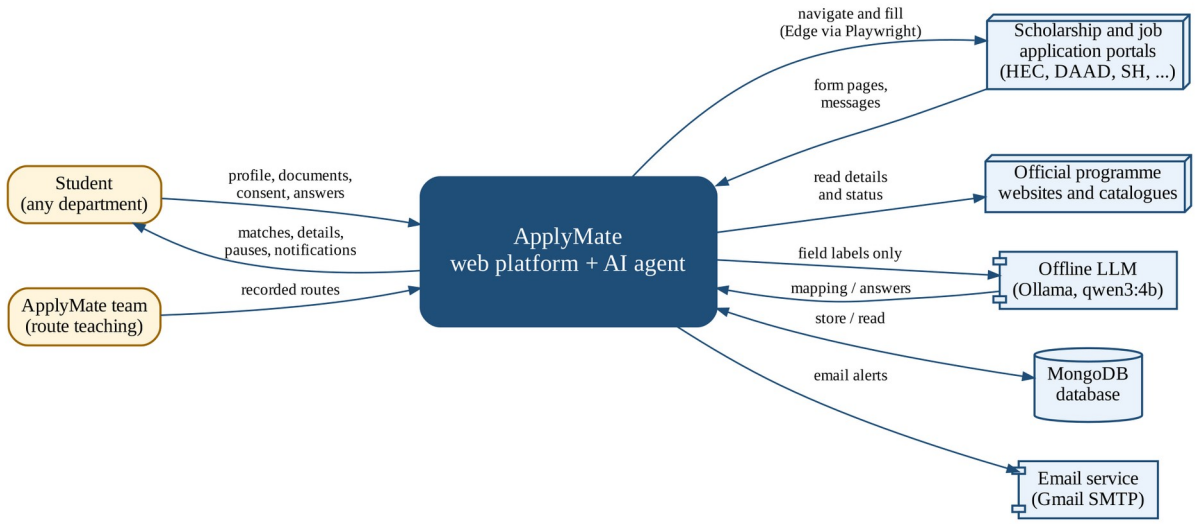


Figure 1: Context diagram of ApplyMate

3.1. Hardware Interfaces

ApplyMate does not control any special hardware. It runs on standard computers and uses the following through the operating system:

- Keyboard, mouse or touch screen for using the web interface.
- A display for the browser window that the agent opens, so the student can watch the agent work and take over when it pauses.
- Local disk storage for uploaded documents (PDF, JPG or PNG), the agent's browser profile and run logs.
- CPU, and optionally a GPU, used by Ollama to run the language model.

No printers, scanners, cameras or other devices are required. Scanned CVs are handled as image or PDF files the student uploads.

3.2. Software Interfaces

The software components ApplyMate connects to, and the data exchanged with each, are:

Component (version)	Purpose and data exchanged
MongoDB 6+ via Mongoose 8	Stores users, profiles, opportunities, application runs, tracker entries and notifications as JSON documents.

Component (version)	Purpose and data exchanged
Ollama with qwen3:4b	Local HTTP API. The server checks that Ollama is running before a run and starts it on the same computer if needed. ApplyMate sends a field label and a list of possible profile keys and receives a JSON answer naming the matching key. It is also used to read CV text and to answer chatbot questions from retrieved listings. Student values are never sent when mapping fields.
Playwright 1.6x with Microsoft Edge / Chrome	Opens the official website, reads the page structure (fields, labels, buttons, dialogs, frames), and types, selects, clicks and uploads on the student's behalf.
Official programme websites	Read over HTTPS with Cheerio to extract summaries, requirements, documents, deadlines and apply links, and to check whether a call is open.
Application portals (HEC Scholarship Portal, DAAD portal, Stipendium Hungaricum and others)	Used through the browser exactly as a student would. No private APIs are called.
pdf.js, Mammoth, Tesseract.js	Extract text from PDF, Word and scanned CVs for profile import.
Nodemailer with an SMTP server	Sends email notifications such as "the agent is waiting for you", deadline reminders, and the 6-digit code for resetting a forgotten password.
bcryptjs, jsonwebtoken	Hash passwords and issue signed session tokens.

Inside ApplyMate, the React client talks to the Express server through a REST API with JSON messages, under routes for authentication, profile, opportunities, applications, tracker, notifications, dashboard and chat. The server starts the agent for a run and shares its progress with the client through the stored application record: the run log, the current pause and the final result.

3.3. Communications Interfaces

ApplyMate uses the following communication methods:

- HTTP/HTTPS between the browser and the ApplyMate server, with JSON request and response bodies. In deployment the connection must use HTTPS.
- Each API request after login carries the JWT in the Authorization header. Requests without a valid token are rejected.
- HTTPS from the agent's browser to external portals and programme websites. ApplyMate never weakens certificate checks.
- Local HTTP between the server and Ollama on the same machine, so model traffic does not leave the computer.
- SMTP over TLS (port 465) for email notifications.
- File uploads from the student to ApplyMate use multipart form data, limited to document types the profile accepts.

While a run is active, the client asks the server for the run's state every few seconds, so the student sees each step and each pause almost immediately. When the student answers a pause, the client sends the answer to the server, and the agent continues.

4. Functional Requirements

4.1. Functional Hierarchy

ApplyMate is divided into seven modules. The diagram shows each module and its sub-functions, and the table after it lists the functional requirements of each module. Requirements marked "(planned)" are designed but will be completed before the final defense. Section 4.2 describes how users carry out these functions through use cases.

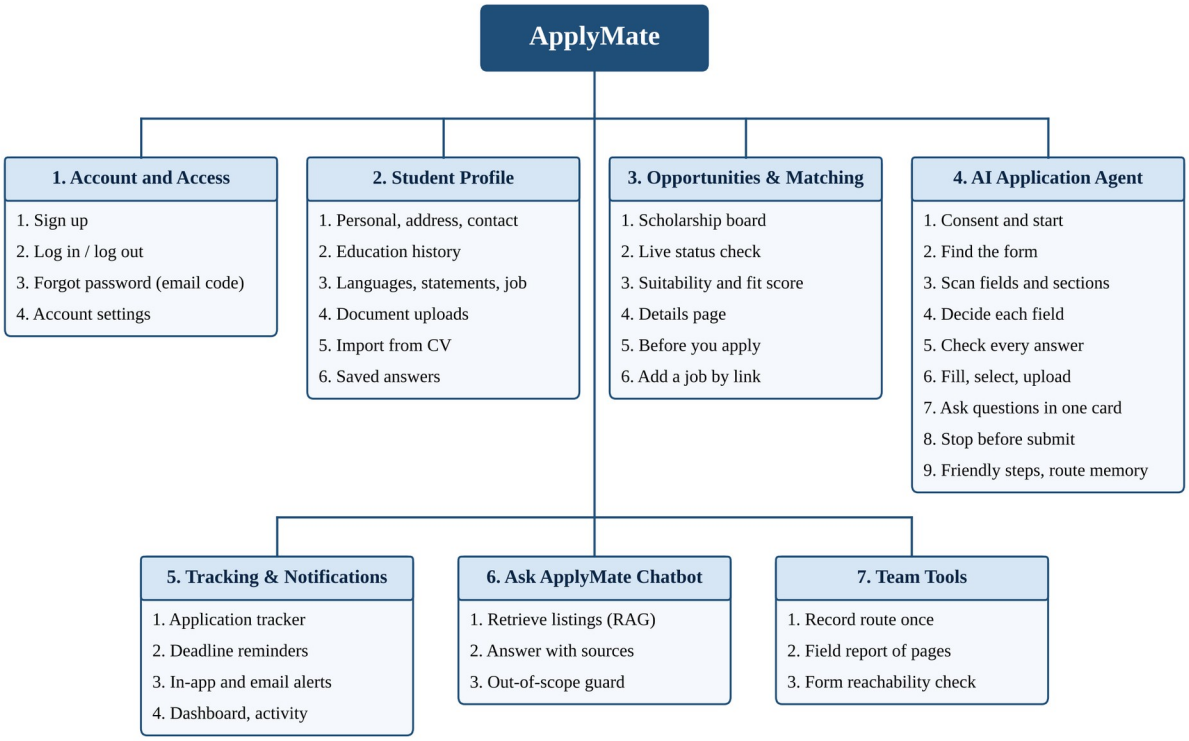


Figure 2: Functional hierarchy of ApplyMate

ID	Functional requirement
FR-1.1	The system shall let a new student sign up with name, email and password, and shall reject an email that is already registered.
FR-1.2	The system shall let a registered student log in and log out, and keep the session for up to seven days.
FR-1.3	The system shall let a student who forgot the password reset it with a 6-digit code sent by email, valid for 15 minutes and limited to a few tries. The answer shall be the same whether the email is registered or not.
FR-1.4	The system shall provide account settings to change the name and password, turn email notifications on or off, and log out.
FR-2.1	The system shall let the student create and edit a profile with personal, address, education, languages, statements, job and preference sections.

ID	Functional requirement
FR-2.2	The system shall store the education history level by level (Matric, Intermediate, Bachelor's, Master's), with board or university, programme, dates, marks or CGPA, and an "in progress" option.
FR-2.3	The system shall let the student upload, replace and delete documents such as CV, CNIC copy, photo, transcript, degree, passport and language certificate.
FR-2.4	The system shall read a CV in PDF or Word format, including scanned CVs through OCR, and suggest profile values that the student reviews before saving.
FR-2.5	The system shall save the student's answers to questions that forms asked (text, choices and documents), and reuse them on every form that asks the same question. The student can change them later.
FR-2.6	(planned) The profile shall hold lists that many international forms ask for: other languages with CEFR levels, two referees (name, position, institution, email), and the language of instruction of the highest degree.
FR-3.1	The system shall provide a curated catalogue of international scholarships and check each programme's current status (open, opens soon, closed, deadline per course).
FR-3.2	The system shall match each opportunity against the student's level, field of study and stated requirements, mark it as suitable or not, show a fit score with reasons, and show suitable opportunities first.
FR-3.3	The system shall show a details page with summary, requirements, documents, deadlines, funding and how to apply, read from the programme's official website.
FR-3.4	The system shall let the student add a job by pasting its link, and read the title and city from the link.
FR-3.5	The scholarships page shall show a board with three columns: open now (nearest deadline first), opening soon (earliest opening first) and deadline passed (with when the next cycle usually opens). Search and filters apply to all columns.
FR-3.6	The details page shall show a "Before you apply" checklist: documents the programme needs compared with the student's uploads, missing profile details, and the questions this form asked in earlier runs, which the student can answer there.
FR-4.1	The system shall ask for the student's consent before each agent run, and shall warn when the programme is closed this cycle.
FR-4.2	The agent shall reach the application form by itself, following a recorded route, a previously learned route, or the most relevant links on the official site, without leaving that site once a route is known.
FR-4.3	The agent shall detect field types on each page, including text, date, dropdown, searchable dropdown, radio, checkbox, file upload, fields inside pop-up dialogs, frames and shadow DOM.
FR-4.4	The agent shall decide each field from the student's saved answer, general label rules, or the offline model, in that order. The model may choose a profile key, give a short answer worked out from general facts about the student, or say that the student must be asked.
FR-4.9	The agent shall check every AI decision before typing it: the decision must belong to the right question, the chosen data must fit the question's topic, and the value must have the right shape (an email address only in an email field, a phone number only in a phone field). A decision that fails is asked again or asked of the student.

ID	Functional requirement
FR-4.10	The agent shall never write essays, motivation statements or abstracts, never answer preference questions (university choices, track, how the student heard about the programme), never guess optional fields, and never use the student's own name, email or phone inside referee or employer sections.
FR-4.11	The agent shall read the section headings of long forms, so that repeated questions ("Full name" for two referees, "First choice" for two semesters) are told apart, and shall fill numbered fields ("Language 1", "Referee 2") only from the matching item of a list in the profile.
FR-4.12	The agent shall collect all questions on a page that only the student can answer and ask them together in one card on the run page, in plain words. Required questions stop the agent; optional ones do not.
FR-4.5	The agent shall fill and verify each field, try different date formats when the site rejects one, choose dropdown options by meaning, and add each missing degree through the portal's "Add Education" pop-up.
FR-4.6	The agent shall pause and explain the reason for login, passwords, codes, CAPTCHAs, essays, declarations, uploads on real websites, missing data, page errors, and pages that do not move on.
FR-4.7	The agent shall never click a final submit button. "Save progress" and "Save draft" buttons are not treated as the next step or as submit. It shall stop on the last page and ask the student to review and submit.
FR-4.8	The system shall show the run to the student in short, friendly steps ("Found the application form", "Filled your name", "Needs you: 2 questions"), keep the full technical log only for the team, and remember the route to a form for future runs.
FR-5.1	The system shall let the student save opportunities to a tracker and move them through stages: saved, in progress, ready, submitted, interview, accepted or rejected.
FR-5.2	The system shall remind the student about tracked deadlines due within seven days.
FR-5.3	The system shall notify the student in the app, and by email if enabled in the settings, when the agent is waiting, when a form is ready to submit, when a run fails, and for reminders. Problems are explained in everyday words with a "Try again" option.
FR-6.1	The chatbot shall answer questions about listed opportunities and the student's applications using only retrieved records, name the listings it used, and politely refuse out-of-scope questions.
FR-7.1	The system shall let a team member record the route to a programme's application form once (for portals that hide the form behind several dashboard pages), and save it for the agent together with a report of the form's field names.
FR-7.2	The system shall provide a check, run by the team, that opens every open scholarship and reports whether its application form is reachable, needs sign-in first, or was not found, without filling anything. The result is shown as a badge on the scholarship.

4.2. Use Cases

The use case diagram shows the five actors of ApplyMate and the eighteen use cases they take part in. The student is the main actor. The ApplyMate team records routes to application forms. The application portal, the offline language model and the email service are external systems that take

part in some use cases. "Apply with Agent" always includes "Respond to Agent Pause", because every run ends with at least one pause (the final review), and "Import Profile from CV" extends "Manage Profile". "Reset Forgotten Password" extends "Log In", and "Answer Form Questions" is included by "Respond to Agent Pause" whenever a page asks something only the student knows.

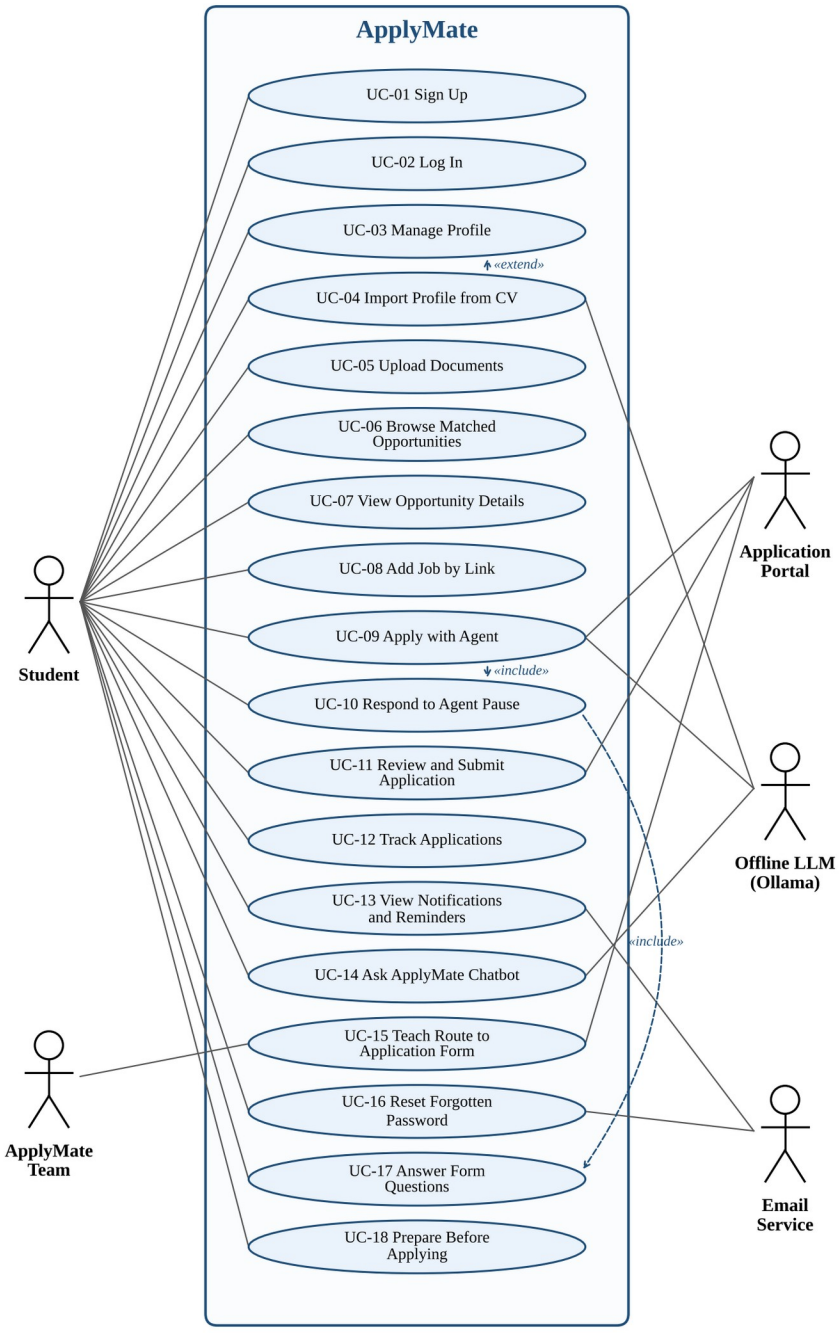


Figure 3: Use case diagram of ApplyMate

4.2.1. UC-01: Sign Up

A new student creates an ApplyMate account.

UC-01: Sign Up		
Use case Id:	UC-01	
Actors: Student (initiator)		
Feature: Account and Access (FR-1.1)		
Pre-condition:	The student is not logged in and has internet access.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens ApplyMate and clicks "Sign up".	Shows the sign-up form.
2.	Enters full name, email and password and submits.	Checks that all fields are filled, the email is valid and the password is long enough.
3.		Checks that the email is not already registered, hashes the password, creates the account with an empty profile, and logs the student in.
4.		Opens the dashboard and suggests completing the profile or importing a CV.
Alternate Scenarios:		
2a: A field is empty or invalid. The system shows the problem next to the field and keeps the entered data. 3a: The email is already registered. The system says so and offers the login page.		
Post Conditions		
Step#	**Description**	
1.	A new account exists and the student is logged in with a session token.	
Use Case Cross referenced	UC-02 Log In, UC-03 Manage Profile	

4.2.2. UC-02: Log In

A registered student signs in to ApplyMate and later signs out.

UC-02: Log In		
Use case Id:	UC-02	
Actors: Student (initiator)		
Feature: Account and Access (FR-1.2)		
Pre-condition:	The student has an ApplyMate account.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens the login page and enters email and password.	Checks the email and compares the password with the stored hash.

2.		Issues a session token valid for seven days and opens the dashboard with suitable opportunities, tracker and notifications.
3.	Later clicks "Log out".	Removes the session token from the browser and returns to the landing page.
Alternate Scenarios:		
1a: The email or password is wrong. The system shows one general message ("Invalid email or password") without saying which part was wrong. 2a: The token has expired. The system asks the student to log in again.		
Post Conditions		
Step#	**Description**	
1.	The student is logged in, or logged out after step 3.	
Use Case Cross referenced	UC-01 Sign Up	

4.2.3. UC-03: Manage Profile

The student creates and keeps the single profile that every application is filled from.

UC-03: Manage Profile		
Use case Id:	UC-03	
Actors: Student (initiator)		
Feature: Student Profile (FR-2.1, FR-2.2)		
Pre-condition:	The student is logged in.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens "Profile".	Shows the profile sections: personal, address, education, education history (Matric, Intermediate, Bachelor's, Master's), languages, statements, job and preferences, with a completion indicator.
2.	Fills or edits fields in a section, for example adds the Intermediate board, marks and dates.	Shows hints for fields that portals often ask for, such as "leave middle name empty if you have none".
3.	Clicks "Save".	Validates formats (email, dates, CNIC pattern, numbers), saves the section and updates the completion indicator and the opportunity matches.
Alternate Scenarios:		
1a: The student chooses "Import from CV" instead. The flow continues in UC-04. 3a: A value has the wrong format. The system marks the field and does not save that section until it is corrected.		
Post Conditions		
Step#	**Description**	

1.	The profile is saved and is used for matching and by the agent in later runs.
Use Case Cross referenced	UC-04 Import Profile from CV, UC-05 Upload Documents, UC-06 Browse Matched Opportunities, UC-09 Apply with Agent

4.2.4. UC-04: Import Profile from CV

The student starts or updates the profile from an existing CV, reviewing every suggestion before it is saved.

UC-04: Import Profile from CV		
Use case Id:	UC-04	
Actors: Student (initiator), Offline LLM		
Feature: Student Profile (FR-2.4)		
Pre-condition:	The student is logged in and has a CV as a PDF or Word file.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Chooses "Import from CV" and uploads the file.	Extracts the text locally: pdf.js for PDF, Mammoth for Word, and Tesseract OCR when the PDF is a scanned image.
2.		Finds clear items such as email, phone and CNIC with patterns, and asks the offline model to suggest the remaining profile fields from the text.
3.		Shows every suggested value next to the current value, so nothing is saved yet.
4.	Reviews the suggestions, unticks or edits any of them and clicks "Apply".	Saves only the accepted values to the profile and sends a notification that the CV was imported.
Alternate Scenarios:		
1a: The file type is not supported or the file is empty. The system shows an error and asks for a PDF or Word file. 2a: The language model is not running. The system still applies the pattern-based suggestions and says that AI suggestions are unavailable.		
Post Conditions		
Step#	**Description**	
1.	The profile contains the values the student accepted. The CV is also stored as the student's CV document.	
Use Case Cross referenced	Extends UC-03 Manage Profile	

4.2.5. UC-05: Upload Documents

The student stores the documents that application forms usually ask for.

UC-05: Upload Documents

Use case Id:	UC-05	
Actors: Student (initiator)		
Feature: Student Profile (FR-2.3)		
Pre-condition:	The student is logged in.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens the Documents section and chooses a document type, such as transcript or passport.	Shows the accepted file types and the current file, if any.
2.	Selects a file and uploads it.	Checks the type and size, stores the file in the student's private storage and links it to the profile.
3.	Optionally deletes or replaces a document.	Removes or replaces the stored file and updates the profile.
Alternate Scenarios:		
2a: The file is too large or of the wrong type. The system rejects it with a clear message.		
Post Conditions		
Step#	**Description**	
1.	The document is available to the agent for upload fields on application forms (only after the student confirms the upload on real websites).	
Use Case Cross referenced	UC-03 Manage Profile, UC-09 Apply with Agent	

4.2.6. UC-06: Browse Matched Opportunities

The student sees the scholarships and jobs that suit their own profile, best matches first.

UC-06: Browse Matched Opportunities		
Use case Id:	UC-06	
Actors: Student (initiator)		
Feature: Opportunities and Matching (FR-3.1, FR-3.2)		
Pre-condition:	The student is logged in. A profile with at least level and field of study gives better matches.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens "Scholarships" or "Jobs".	Loads the catalogue, refreshes each programme's status if it is out of date, and scores every listing against the profile.

2.		For scholarships, shows a board with three columns: open now, opening soon and deadline passed, each with a count. Cards show the fit score, deadline or opening date, and, when checked, whether the form is reachable. "Apply with agent" appears only on open scholarships.
3.	Switches the filter to "All" or searches by keyword, country or level.	Shows the requested listings. Unsuitable ones carry a short line explaining why, for example "needs two years of work experience".
4.	Saves a listing to the tracker or opens it.	Adds it to the tracker (UC-12) or opens the details page (UC-07).
Alternate Scenarios:		
1a: The profile is nearly empty. The system shows all listings and suggests completing the profile for better matching. 2a: No listing suits the student. The system says so and shows the closest ones with their blockers.		
Post Conditions		
Step#	**Description**	
1.	The student sees the opportunities that fit the profile.	
Use Case Cross referenced	UC-03 Manage Profile, UC-07 View Opportunity Details, UC-12 Track Applications	

4.2.7. UC-07: View Opportunity Details

The student reads everything needed about one opportunity before deciding to apply.

UC-07: View Opportunity Details		
Use case Id:	UC-07	
Actors: Student (initiator)		
Feature: Opportunities and Matching (FR-3.3)		
Pre-condition:	The student is logged in and has selected an opportunity.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens the opportunity.	Shows cached details if they are recent; otherwise reads the programme's home page and up to three related pages ("How to apply", "Eligibility") from the official website.
2.		Shows the summary, requirements and eligibility, documents to prepare, dates and deadlines, funding and how to apply, together with the fit score and a link to the official page.
3.		If the route to the form is already known, shows it, for example "Hungarian Scholarship Program, found by the agent".

4.	Clicks "Read the website again" (optional).	Reads the official pages again and updates the details.
Alternate Scenarios:		
1a: The website cannot be reached or lists nothing readable. The system shows the information it has from the catalogue and advises opening the official page.		
Post Conditions		
Step#	**Description**	
1.	The student knows what the programme requires before applying.	
Use Case Cross referenced	UC-06 Browse Matched Opportunities, UC-09 Apply with Agent	

4.2.8. UC-08: Add Job by Link

The student adds a job found on a job site that ApplyMate cannot read automatically.

UC-08: Add Job by Link		
Use case Id:	UC-08	
Actors: Student (initiator)		
Feature: Opportunities and Matching (FR-3.4)		
Pre-condition:	The student is logged in and has copied a job link from Rozee.pk or another job site.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens "Jobs" and pastes the link into "Add a job by link".	Validates the link.
2.		Reads the job title, company and city from the link itself, without loading the protected page, and creates a job listing for the student.
3.	Edits the title or city if needed and saves.	Shows the job in the student's list, where it can be tracked or applied to.
Alternate Scenarios:		
1a: The text is not a valid link. The system asks for a full job link. 2a: The link has no readable title. The system creates the listing with the website name and asks the student to type the title.		
Post Conditions		
Step#	**Description**	
1.	The job appears in the student's jobs list.	
Use Case Cross referenced	UC-12 Track Applications, UC-09 Apply with Agent	

4.2.9. UC-09: Apply with Agent

The main use case of ApplyMate: the agent opens the official website, reaches the application form by itself and fills it from the profile, involving the student only when necessary.

UC-09: Apply with Agent		
Use case Id:	UC-09	
Actors: Student (initiator), Application Portal, Offline LLM		
Feature: AI Application Agent (FR-4.1 to FR-4.8)		
Pre-condition:	The student is logged in, the profile is reasonably complete, the agent machine has Edge or Chrome and Ollama running, and the opportunity has an official website.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Clicks "Apply with agent" on the details page.	Shows a consent dialog explaining what the agent will and will not do, and a warning if the call is closed.
2.	Gives consent.	Creates an application run, opens a browser window on the official website and shows the live run page.
3.		Finds the application form: follows the recorded route or the remembered route, or scores the site's links ("Apply", "Scholarships") and follows the best ones, staying on the same site. If the portal shows a sign-in page, it pauses for the student to log in (UC-10).
4.		On each form page, scans every field, dropdown, radio group, pop-up and frame, and records its label and type.
5.		Decides each field: the student's saved answer first, then general label rules, then the offline model, which sees the question, the possible profile keys and general facts about the student, never identifiers such as CNIC, phone or passport number. Every decision is checked before use. Essays and preference questions always go to the student.
6.		Fills each field, checks that the value stuck, and closes harmless "saved successfully" messages. For lists such as education history it opens "Add Education" for every missing degree and fills the pop-up with that degree's data.
7.		Clicks the form's own Next or Continue and repeats steps 4 to 6 on the next page. It never clicks a button whose meaning is final submission.

8.		On the last page, pauses with "All fields are filled. Please review the form and submit it yourself", notifies the student and saves the route for future runs.
Alternate Scenarios:		
3a: No link clearly says "Apply". The agent uses application links it read earlier on the programme's website, and otherwise lets the offline model read all the links on the page and choose the most likely way to the form. It keeps searching on the programme's own website until it reaches the form or the portal's sign-in page, and saves the way it found for every student after. 3b: The site opens with a start page ("I want to fill in a new form" and Next). The agent keeps the selected choice and clicks Next. 5a: Required questions remain that only the student can answer. The agent asks them all at once in a card (UC-17) and continues with the answers. 4a: The page has a CAPTCHA, password, code, essay, declaration or upload on a real website. The agent pauses (UC-10). 6a: A value is rejected by the site, for example a date format. The agent tries other formats, then pauses with the site's error message. 7a: Next is clicked twice and the page does not move on, as when a portal's eligibility check blocks the student. The agent pauses with "the page did not move on" instead of looping. *a: The student clicks "Stop the agent" at any time. The run ends and the browser stays open as it is.		
Post Conditions		
Step#	**Description**	
1.	The application form is filled up to the final submit, or the run is paused or stopped with a clear reason. The run log and result are saved and the tracker entry is updated.	
Use Case Cross referenced	Includes UC-10 Respond to Agent Pause; leads to UC-11 Review and Submit; uses UC-03, UC-05 and UC-15	

4.2.10. UC-10: Respond to Agent Pause

The student takes over for a moment when the agent needs something only a person can provide, then hands control back.

UC-10: Respond to Agent Pause		
Use case Id:	UC-10	
Actors: Student (initiator)		
Feature: AI Application Agent (FR-4.6)		
Pre-condition:	An agent run is active and the agent has paused.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.		Shows a "Your turn" box on the run page with the reason and what to do, and sends an in-app notification (and email, if enabled).

2.	Reads the message and acts in the browser window: logs in, types a code, solves a CAPTCHA, writes an essay, ticks a declaration, or fills fields the profile does not have.	Waits without changing the page.
3.	Clicks "I've done it, continue". When the pause is about questions, answers them in the card on the run page instead (UC-17).	Scans the page again, continues from where it stopped and records the pause and answer in the log.
Alternate Scenarios:		
3a: The student clicks "Stop the agent". The run ends with the form as it is. 3b: Nothing has changed since the last pause. The agent says so and stops, so it never loops on the same problem. 3c: The student saves a missing value to the profile from the pause. Future runs fill it automatically.		
Post Conditions		
Step#	**Description**	
1.	The agent continues the run, or the run ends at the student's request.	
Use Case Cross referenced	Included by UC-09 Apply with Agent; includes UC-17 Answer Form Questions	

4.2.11. UC-11: Review and Submit Application

The student checks the filled form and submits it personally, because the agent never submits.

UC-11: Review and Submit Application		
Use case Id:	UC-11	
Actors: Student (initiator), Application Portal		
Feature: AI Application Agent (FR-4.7)		
Pre-condition:	The agent has paused with "ready to submit".	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens the browser window and reviews every page of the filled form.	Keeps the browser open and does not touch the page.
2.	Corrects anything on the portal if needed and clicks the portal's own submit button.	Not involved: submission happens directly between the student and the portal.
3.	Marks the application as submitted in ApplyMate.	Moves the tracker entry to "Submitted" and records the date.
Alternate Scenarios:		
2a: The student decides not to submit yet. The application stays "Ready" in the tracker, and the portal usually keeps the draft.		
Post Conditions		

Step#	Description
1.	The application is submitted by the student, and ApplyMate's tracker reflects it.
Use Case Cross referenced	UC-09 Apply with Agent, UC-12 Track Applications

4.2.12. UC-12: Track Applications

The student follows the progress of every application in one place.

UC-12: Track Applications		
Use case Id:	UC-12	
Actors: Student (initiator)		
Feature: Tracking and Notifications (FR-5.1)		
Pre-condition:	The student is logged in.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens "Tracker".	Shows saved opportunities grouped by stage: saved, in progress, ready, submitted, interview, accepted, rejected.
2.	Moves an entry to another stage, sets a personal deadline or adds a note.	Saves the change and adds it to the entry's history.
3.	Removes an entry.	Deletes the entry after confirmation.
Alternate Scenarios:		
2a: An agent run changes the stage automatically, for example to "In progress" or "Ready". The tracker shows the new stage on the next refresh.		
Post Conditions		
Step#	**Description**	
1.	The tracker reflects the student's current progress on each opportunity.	
Use Case Cross referenced	UC-06, UC-09, UC-11, UC-13	

4.2.13. UC-13: View Notifications and Reminders

The student is told when the agent needs attention, when a form is ready, and when a deadline is close.

UC-13: View Notifications and Reminders	
Use case Id:	UC-13
Actors: Student (initiator), Email Service	
Feature: Tracking and Notifications (FR-5.2, FR-5.3)	
Pre-condition:	The student is logged in. Email requires SMTP to be configured and email notifications to be turned on.
Scenarios	

Step#	Action	Software Reaction
1.		Creates notifications when the agent is waiting, a form is ready to submit, a run fails, a CV is imported, or a tracked deadline is within seven days (at most once a day per entry).
2.		Shows an unread count in the header and, if enabled, sends the same notification by email.
3.	Opens the notifications panel and clicks one.	Marks it as read and opens the related page, such as the run page.
4.	Changes email settings or sends a test email.	Saves the settings or sends a test message and reports whether it was delivered.
Alternate Scenarios:		
2a: Email sending fails. The in-app notification still appears and the failure is logged.		
Post Conditions		
Step#	**Description**	
1.	The student is aware of anything that needs attention.	
Use Case Cross referenced	UC-09, UC-10, UC-12	

4.2.14. UC-14: Ask ApplyMate Chatbot

The student asks questions about the opportunities on ApplyMate and gets answers based only on the listed information.

UC-14: Ask ApplyMate Chatbot		
Use case Id:	UC-14	
Actors: Student (initiator), Offline LLM		
Feature: Ask ApplyMate Chatbot (FR-6.1)		
Pre-condition:	The student is logged in and the language model is running.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Opens the chat widget and asks a question, for example "Which scholarships fit me in Germany?".	Retrieves the listings that match the question and the student's match scores, plus the student's tracker entries.
2.		Asks the offline model to answer only from those records, and to report whether the question is in scope.
3.		Shows the answer with the listings it used as links. Only basic profile details (level, field, city, degree) are used; never CNIC, phone or documents.
4.	Asks a short follow-up such as "What about its deadline?".	Uses the previous question as context and answers.

Alternate Scenarios:	
2a: The question is out of scope, such as general homework. The system replies with a fixed polite message explaining what it can help with. 2b: The model is not running. The system says the assistant is unavailable.	
Post Conditions	
Step#	**Description**
1.	The student receives an answer grounded in ApplyMate's listings.
Use Case Cross referenced	UC-06 Browse Matched Opportunities

4.2.15. UC-15: Teach Route to Application Form

A team member shows ApplyMate the way to a programme's application form once, so the agent can follow it for every student.

UC-15: Teach Route to Application Form		
Use case Id:	UC-15	
Actors: ApplyMate team member (initiator), Application Portal		
Feature: Route Teaching (FR-7.1)		
Pre-condition:	The programme is in the catalogue and the team member has a test account on its portal.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Runs the teaching tool for the programme.	Opens a browser at the programme's start address.
2.	Logs in to the portal personally and presses Enter.	Starts recording clicks and choices. Passwords are never recorded.
3.	Clicks to the first page of the application form as a student would, for example "Hungarian Scholarship Program".	Records each step by its visible text.
4.	On each form page presses Enter, then moves on with the form's own Next.	Reads the page's field names (never values) for the field report.
5.	Types "done" and confirms the route.	Saves the route for the agent and the field report for the team.
Alternate Scenarios:		
5a: The member removes wrong steps by number before saving. 5b: The member answers "no" and nothing is saved.		
Post Conditions		
Step#	**Description**	
1.	The agent can follow the recorded route for every student applying to that programme.	
Use Case Cross referenced	Used by UC-09 Apply with Agent	

4.2.16. UC-16: Reset Forgotten Password

A student who forgot the password sets a new one with a code sent by email.

UC-16: Reset Forgotten Password		
Use case Id:	UC-16	
Actors: Student (initiator), Email Service		
Feature: Account and Access (FR-1.3)		
Pre-condition:	The student has an account but has forgotten the password. Email sending is configured on the server.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Clicks "Forgot password?" on the login page and enters the email address.	Shows the same message whether or not the email is registered, so nobody can find out who has an account.
2.		If the account exists, sends a 6-digit code by email. Only a hash of the code is stored; it expires after 15 minutes.
3.	Enters the code and a new password.	Checks the code and the number of tries, saves the new password as a hash, and asks the student to log in.
Alternate Scenarios:		
1a: A new code is asked for within one minute. The system does not send another email yet. 3a: The code is wrong or expired, or there were too many tries. The system says so and offers to send a new code. 1b: Email is not set up on the server. The system says that password reset is not available and to contact the team.		
Post Conditions		
Step#	**Description**	
1.	The student can log in with the new password.	
Use Case Cross referenced	Extends UC-02 Log In	

4.2.17. UC-17: Answer Form Questions

The student answers, in plain words, the questions a form asked that the profile could not answer. The answers are saved and reused on every form.

UC-17: Answer Form Questions	
Use case Id:	UC-17
Actors: Student (initiator)	
Feature: Student Profile and AI Application Agent (FR-2.5, FR-4.12)	

Pre-condition:	The agent paused with questions it cannot answer, or the student opened a scholarship whose form asked questions in earlier runs.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.		Shows the questions in plain words, one input each: a choice list or Yes/No for choice questions, a text box for text, an upload button for documents. Required questions are marked; optional ones are folded away.
2.	Answers the questions and uploads any documents asked for.	Uploads each document straight away and checks that every required question has an answer. No option is pre-selected.
3.	Clicks "Save answers and continue" (or "Save my answers" on the details page).	Saves the answers in the profile, hands them to the agent, which fills them at once, and reuses them on every form that asks the same question.
Alternate Scenarios:		
2a: The student leaves a required question empty. The agent asks again for that one only. 3a: The student wants to change an earlier answer. On the details page, "Change my answers" shows all saved answers for this form.		
Post Conditions		
Step#	**Description**	
1.	The answers are in the form and saved in the profile.	
Use Case Cross referenced	Included by UC-10 Respond to Agent Pause; used by UC-18	

4.2.18. UC-18: Prepare Before Applying

Before starting the agent, the student sees everything this programme's form needs and adds what is missing, so the agent does not have to stop.

UC-18: Prepare Before Applying		
Use case Id:	UC-18	
Actors: Student (initiator)		
Feature: Opportunities and Matching (FR-3.6)		
Pre-condition:	The student is logged in and has opened a scholarship's details page.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.		Shows the "Before you apply" checklist: each document the programme needs, ticked if it is in the profile; profile details most forms ask for that are still empty; and the questions this form asked in earlier runs.

2.	Uploads missing documents in the profile, fills missing details, and answers the open questions (UC-17).	Updates the checklist and shows "you have everything the agent needs" when nothing is missing.
3.	Clicks "Apply with agent".	Starts the run (UC-09), which now needs far fewer pauses.
Alternate Scenarios:		
1a: Nobody has applied to this form through ApplyMate yet. Only documents and profile details are listed; the form's questions appear after the first run.		
Post Conditions		
Step#	**Description**	
1.	The student is ready, and the agent can fill the form without stopping for missing data.	
Use Case Cross referenced	UC-07 View Opportunity Details, UC-09 Apply with Agent, UC-17 Answer Form Questions	

5. Non-functional Requirements

5.1. Performance Requirements

ApplyMate must meet the following performance requirements on the recommended machine (Section 2.6):

ID	Requirement
NFR-P1	Pages of the web interface should load within 3 seconds on a normal broadband connection.
NFR-P2	Opportunity matching for one student should complete within 2 seconds for the whole catalogue.
NFR-P3	The agent should scan a typical form page (up to about 50 fields) within 5 seconds.
NFR-P4	Most fields shall be decided by saved answers and rules without the model. Essays, preferences and optional fields never reach the model. A single language-model request shall time out after 90 seconds and be retried once, so the run never hangs.
NFR-P5	The live run page should show a new agent step or pause within 5 seconds of it happening.
NFR-P6	Programme details read from a website shall be cached and reused for several days, so each official website is read at most once in that period.
NFR-P7	The agent shall wait at most about 8 seconds for a covered button before retrying once and pausing with an explanation.
NFR-P8	The system should support at least 50 registered students and several simultaneous agent runs on one server, with each run in its own browser.
NFR-P9	On a long one-page form (about 90 fields), the time spent waiting for the model should stay under one minute on the recommended laptop. On the real NOHA form it fell from 479 seconds to 43 seconds after the model was asked fewer and shorter questions.
NFR-P10	(planned) The agent should remember each form's decisions (question to profile key or "ask", never values), so that a second run on the same form needs almost no model calls.

5.2. Safety Requirements

The main risk in ApplyMate is that the agent does something on a real website that the student did not intend. The following requirements prevent that harm:

ID	Requirement
NFR-S1	The agent shall never click a final submit button or a button whose text means submit, apply now, finish or send. Submission is always the student's own action.
NFR-S2	The agent shall never accept declarations, terms or consent checkboxes on real websites; it shall pause for the student.
NFR-S3	The agent shall ask the student before uploading any document to a real website.
NFR-S4	The agent shall stay on the programme's own website once a route is known, and shall avoid links to vendors, advertising, pricing or unrelated services.

ID	Requirement
NFR-S5	The agent shall detect when it is stuck (the same page after two Next clicks, or no change since the last pause) and stop with an explanation instead of looping.
NFR-S6	The agent shall only close message pop-ups that contain no fields and do not mention submitting, final, declaration, deleting or removing.
NFR-S7	The agent shall leave a field empty when the profile has no value for it (for example a middle name the student does not have) rather than guessing.
NFR-S8	Every run shall be logged step by step, so the student and the team can see exactly what the agent did.
NFR-S9	The student shall be able to stop the agent at any moment from the run page.
NFR-S10	An AI answer shall never be typed unless it passed the checks of FR-4.9. Answers such as "ask", "none" or a key name shall be rejected.
NFR-S11	The agent shall never write essays, motivation statements or abstracts, and shall never choose preferences for the student (FR-4.10).
NFR-S12	The student's own name, email and phone shall never be filled inside referee, reference, contact person or employer sections.
NFR-S13	"Save progress" and "Save draft" buttons shall not be clicked as a next step, and shall never be treated as the final submit.

5.3. Security Requirements

ApplyMate holds personal data such as CNIC numbers, education records and documents, so it must meet these security and privacy requirements:

ID	Requirement
NFR-SEC1	ApplyMate passwords shall be stored only as bcrypt hashes. Plain passwords shall never be stored or logged.
NFR-SEC2	Every API request except sign-up and login shall require a valid signed JWT. A student shall only be able to read and change their own profile, documents, runs, tracker and notifications.
NFR-SEC3	The agent shall never type, store or see passwords, verification codes or CAPTCHA answers for external portals. The student enters them directly in the browser.
NFR-SEC4	The language model shall receive field labels, the names of profile keys and only general facts about the student (nationality, country and city of residence, degree status and field, languages, years of experience, and the student's own saved answers). It shall never receive the CNIC, passport number, phone number, email address, date of birth or street address; those are filled by code.
NFR-SEC5	All language-model processing shall run locally through Ollama. No profile data or document shall be sent to a cloud AI service.
NFR-SEC6	The chatbot shall use only basic profile details (level, field, city, degree) and never CNIC, phone numbers or documents.
NFR-SEC7	Secrets such as the JWT key, database address and SMTP password shall be kept in an environment file that is never committed to version control. The agent's browser data and uploaded documents shall also be excluded from the repository.

ID	Requirement
NFR-SEC8	Uploaded documents shall be stored in private storage, limited to accepted file types and sizes, and removed when the student deletes them.
NFR-SEC9	ApplyMate shall not attempt to bypass CAPTCHAs, bot protection or access controls of any external website.
NFR-SEC10	In deployment, all traffic between the browser and the server shall use HTTPS.
NFR-SEC11	Password reset codes shall be stored only as hashes, expire after 15 minutes, allow a limited number of tries, and the reset page shall not reveal whether an email address is registered.

5.4. User Documentation

The following documentation will be delivered with ApplyMate:

- A user guide for students covering sign-up, building the profile, importing a CV, uploading documents, finding suitable opportunities, starting the agent, answering its pauses and using the tracker.
- In-app help: the "How this run works" panel on every run page, plain-language pause messages, field hints in the profile, and the consent dialog before each run.
- An installation and setup guide (README) for running the server, client, MongoDB and Ollama, including the environment settings, the browser the agent uses, and a one-click update script for new versions.
- Plain-language messages on the run page: each step and each problem is described in everyday words, with "Try again" when something outside ApplyMate failed, such as a website that did not open.
- A team guide for recording a new route with the teaching tool and adding a programme to the catalogue.
- This SRS and the SDS, as technical references for the supervisor, committee and future developers.

6. References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC/IEEE, "Systems and software engineering: Life cycle processes, Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.

[3] X. Deng et al., "Mind2Web: Towards a Generalist Agent for the Web," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[4] S. Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," in International Conference on Learning Representations (ICLR), 2024.

[5] H. He et al., "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models," in Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

[6] B. Zheng, B. Gou, J. Kil, H. Sun and Y. Su, "GPT-4V(ision) is a Generalist Web Agent, if Grounded," in International Conference on Machine Learning (ICML), 2024.

[7] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[8] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[9] Microsoft, "Playwright documentation." [Online]. Available: https://playwright.dev

[10] Ollama, "Ollama documentation." [Online]. Available: https://ollama.com

[11] MongoDB Inc., "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs/manual/

[12] Meta Open Source, "React documentation." [Online]. Available: https://react.dev

[13] OpenJS Foundation, "Express documentation." [Online]. Available: https://expressjs.com

[14] Higher Education Commission of Pakistan, "HEC Scholarship Portal." [Online]. Available: https://scholarship.hec.gov.pk

[15] Tempus Public Foundation, "Stipendium Hungaricum." [Online]. Available: https://stipendiumhungaricum.hu

[16] DAAD, "Scholarship database." [Online]. Available: https://www2.daad.de/deutschland/stipendium/datenbank/en/

[17] Foreign, Commonwealth and Development Office, "Chevening Scholarships." [Online]. Available: https://www.chevening.org

[18] United States Educational Foundation in Pakistan, "Fulbright programmes." [Online]. Available: https://www.usefp.org

[19] European Education and Culture Executive Agency, "Erasmus Mundus Catalogue." [Online]. Available: https://www.eacea.ec.europa.eu/scholarships/erasmus-mundus-catalogue_en

7. Appendices

Appendix A: Profile data used by the agent

The table lists the profile sections the agent can fill from. Field names are general, not tied to one portal.

Section	Main fields
Personal	Title, first, middle and last name, father's name, gender, date and place of birth, nationality, CNIC, passport number and expiry, marital status, domicile, email, phone
Address	Street, city, province, postal code, country
Education (summary)	Current level, field of study, university, CGPA, graduation year
Education history	For Matric, Intermediate, Bachelor's and Master's: programme title, discipline, board or university, sector, country, city, start and end dates, roll number, obtained and total marks or CGPA and scale, in progress
Languages	Mother tongue, English level and test scores, other languages
Statements	Motivation, career goals and other texts the student wrote and saved
Job	Current employer, position, years of experience
Preferences	Preferred countries, levels and fields, previous scholarships held
Documents	CV, CNIC copy, photo, transcript, degree, passport, language certificate, motivation letter
Saved answers	Answers to questions forms asked that are not in the sections above (text, choices and documents), reused on every form
Lists (planned)	Other languages with CEFR levels; two referees with name, position, institution and email

Appendix B: When the agent pauses

Situation on the page	What the agent does
Sign-in page, password or verification code	Pauses and asks the student to log in or type the code in the browser.
CAPTCHA	Pauses. It never tries to solve it.
Essay, motivation text or free answer not in the profile	Pauses and asks the student to write it.
Declaration, terms or consent checkbox	Pauses. The student ticks it personally.
File upload on a real website	Asks the student to confirm before uploading.
Questions only the student can answer	Asks them all at once in a card on the run page. Answers are saved and reused.
Essay, abstract or preference question	Always asks the student; never written or chosen by the AI.
Error message shown by the portal	Pauses with the portal's own message.

Situation on the page	What the agent does
Page does not move on after Next	Pauses with "the page did not move on" and explains what to check.
Final submit button reached	Stops and asks the student to review and submit.

Appendix C: Implementation status (6 November 2026)

All seven modules exist in our prototype. The table summarises the agent runs on real portals and on our practice portals. Numbers come from the agent's own run logs; nothing was submitted on any real portal.

Portal	Result
HEC Scholarship Portal (Stipendium Hungaricum)	Logged in with the student, followed the recorded route, filled the Profile page and recognised the education history. HEC's own eligibility check then blocked every applicant because the call was not yet active (isScholarshipActive: false); the agent noticed that the page did not move on and paused with an explanation.
Erasmus Mundus EMCL+ (Clinical Linguistics, Formdesk)	Found the application form from the programme's admission page, handled the form tool's start page, and filled all pages: 50 of 50 actions succeeded in about 5 minutes, stopping before "Final Submission". Questions only the student could answer were asked once and saved.
Erasmus Mundus NOHA (one long page, about 90 fields)	After the safety checks and shorter prompts: 93 of 93 actions succeeded in 324 seconds (813 seconds before), with 43 seconds spent waiting for the model (479 seconds before). It stopped to ask for preferences and essays.
Erasmus Mundus EuroPubHealth+ (eMundus)	Reached the portal's sign-in page, where the student signs in personally.
Practice portals (HEC-style, DAAD-style, five local forms)	All filled correctly (for example 17/17 and 37/37 actions), including a form that copies NOHA's patterns: sections, two referees, a calendar date picker, preferences and essays.